Typing

CS 1622

Jarrett Billingsley

Class Announcements

- I may hold some Extra Office Hours on Friday… what time would be best for those of you who think you may need them?

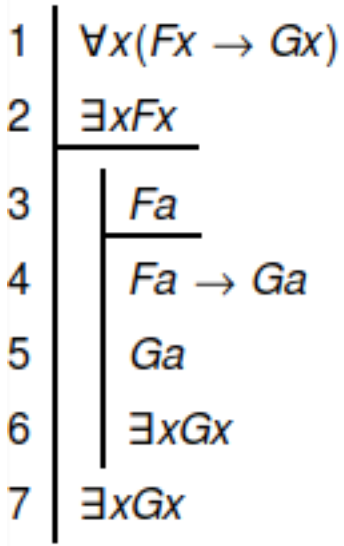
A little Type Theory

Remember what a type is?

- a **type** is:
  - a **set of valid values,** plus:
  - a **set of valid operations on those values**
- think about `int`s:
  - there's a range of valid values of an `int` variable
  - and a set of **operators** and other things you can use on them
- **type theory** is a branch of mathematics
  - it might seem kind of abstract and confusing (as most math does)
  - but it has had **profound impacts** on the design and behaviors of programming languages, especially in the past 30 years
- without rigorous type theory, you get C 😬

I'll Curry YOUR Howards, isomorphically

- remember from the beginning of the term: **programs are proofs.**
- type systems are essentially **proof justifications.**



these justifications are saying, "look, I'm using only **valid operations** on these values!"

1 ∀E
3, 4 →E
5 ∃I
2, 3–6 ∃E

similarly, when you say "this variable is an int," the compiler uses that information to **justify** the steps of your program.

if it can't... **that's a type error!**

Type Safety = Progress + Preservation

- **type safety** means that you know exactly what will happen at every step of your program, because you know the types of everything.

progress means that at every step of computation:

either the program **halts…**

```
int[] a = { 1, 2, 3 };
a[4] = 10; // halts
```

…or there is **exactly one thing that can happen.**

```
int[] a = { 1, 2, 3 };
a[0] = 10; // assigns
```

preservation means that you **know the type** of **every value,** at **every step** of computation. (you never "leave the walls" of the type system.)

Java *is* a type-safe language, so I can't give an example of that.

but what about the most popular language used for programming our operating systems, device drivers, embedded device firmware etc.? 🙃

A very unsafe language

- in C, the type system was an afterthought.

```
float f = 3.4;
int a[10];
a[10] = 0xDEADBEEF;
printf("%f\n", f);
```

this line of code **violates both progress and preservation.**

it violates **progress** because accessing an array out-of-bounds is "undefined behavior" (**UB**), which means "anything could happen."

it violates **preservation** because one of the things that *could* happen is that this line **overwrites the value in f, violating its type.**

depending on the platform and compilation settings, this **printf** might show **3.4** or **-6259853398707798016.000000!**

Safety has to be built-in

- once you step into UB, **you cannot guarantee anything.**
  - it's like writing **a proof with an incorrect step.**
  - all the steps after *may or may not* be invalid!
  - HLLs are a set of abstractions, and type-unsafety lets you **ruin those abstractions entirely.**
- you **can't make C safe.** this has big consequences:
  - you **can't do automatic memory management,** because that requires you to know the types of every value in the program.
  - you **can't prove that a C program is correct,** because there are many "correct" programs which actually violate type safety.
- you **can't be superhuman.** sorry, but no amount of "just write the code correctly" is going to lead to correct C code. mistakes happen.
  - this is *why* languages like Java and Rust exist!

Decidability

- **decidable** means: "can this question be answered in finite time?"
- type-checking is **an algorithm,** so **it *can* be undecidable.**
  - we can make our type system so powerful that we'd have to **solve the halting problem to check our program's types!**
    - which maybe seems like a bad thing, right?
- there's this other related thing that I think is related to decidability?
  - you **can't predict the future.**
  - that means **sometimes you can't know what type something is until runtime, no matter how clever your type system is.**
- so if you want programmers to have that (very useful!) ability, you have to design your type system to allow for it **in a safe way.**
  - we'll see some examples shortly.

Type systems

The primitive types

- a language's **problem domain** is the problems it's designed to solve.
  - most languages try to be "general purpose" and therefore give you relatively few, simple primitive types, like **int**, **char**, **bool**, etc.
- the **hardware** also dictates what primitives are available.
  - e.g. on GPUs, 4-element vectors and 4x4 matrices are primitives!
- **domain-specific languages** (DSLs) are designed to do one job well.
  - these might have many primitive types, including types which don't seem very "primitive" at all.
  - for example, a shell scripting language may have primitive types like files, processes, and terminal emulators.
- the set of primitive types is really a product of **what problems you want your language to be able to solve easily.**

Static vs. Dynamic typing

- remember: **static** ="before runtime;" **dynamic** = "at runtime."

statically typed languages
associate types with *variables*.

```
let mut a = 10;
a = 20;       // ok
a = "hello"; // bad
a = a / 2;    // ok
```

dynamically typed languages
associate types with *values*.

```
let a = 10;   // JS
a = 20;       // ok
a = "hello"; // ok!
a = a / 2;    // bad
```

in dynamically typed languages, **type errors cannot be found until you run the program.**

but this is a *spectrum:* many statically-typed languages *include* dynamically-typed features too!

Dynamic types in static languages

- sometimes it's *useful* not to know the type until runtime.

```
class A { String toString() { return "A"; } }
class B { String toString() { return "B"; } }

Object o;

if(user types A)
  o = new A();
else
  o = new B();

o.toString();
```

this works because of Java's **subtyping** rules and **virtual methods.**

calling **toString()** is a valid operation on any **Object**, and every class instance is an **Object**.

this is a form of *existential polymorphism:* we don't know **what type** it is until runtime, but we *at least* know that it **will** support this operation!

Type coercion ("punning") and conversion

- **type coercion** is when the language will let you use the "wrong" type and it will automatically turn it into the "right" one.
- **type conversion** creates a new value of a different type.
  - some type coercions do a type conversion; some don't!

```java
// Java
int x = 10;
float f = x;
// now f == 10.0
int x = 10;
String s = "x = " + x;
// now s is "x = 10"
```

implicit conversions like these can make code shorter and more readable...

but they can be confusing too:

```javascript
// JavaScript
let x = 10 + "20";
let y = 10 - "20";
// x == "1020"
// y == -10 ?!
```

Strong vs. Weak typing

- different people use “strong” and “weak” to mean different things.

one definition is basically the same as **type safety:** it says that weakly-typed languages let you “step around” the type system, while strongly-typed languages keep you in.

```c
// C
bool b = true;
int* p = (int*)&b;
*p = 700; // wat.
```

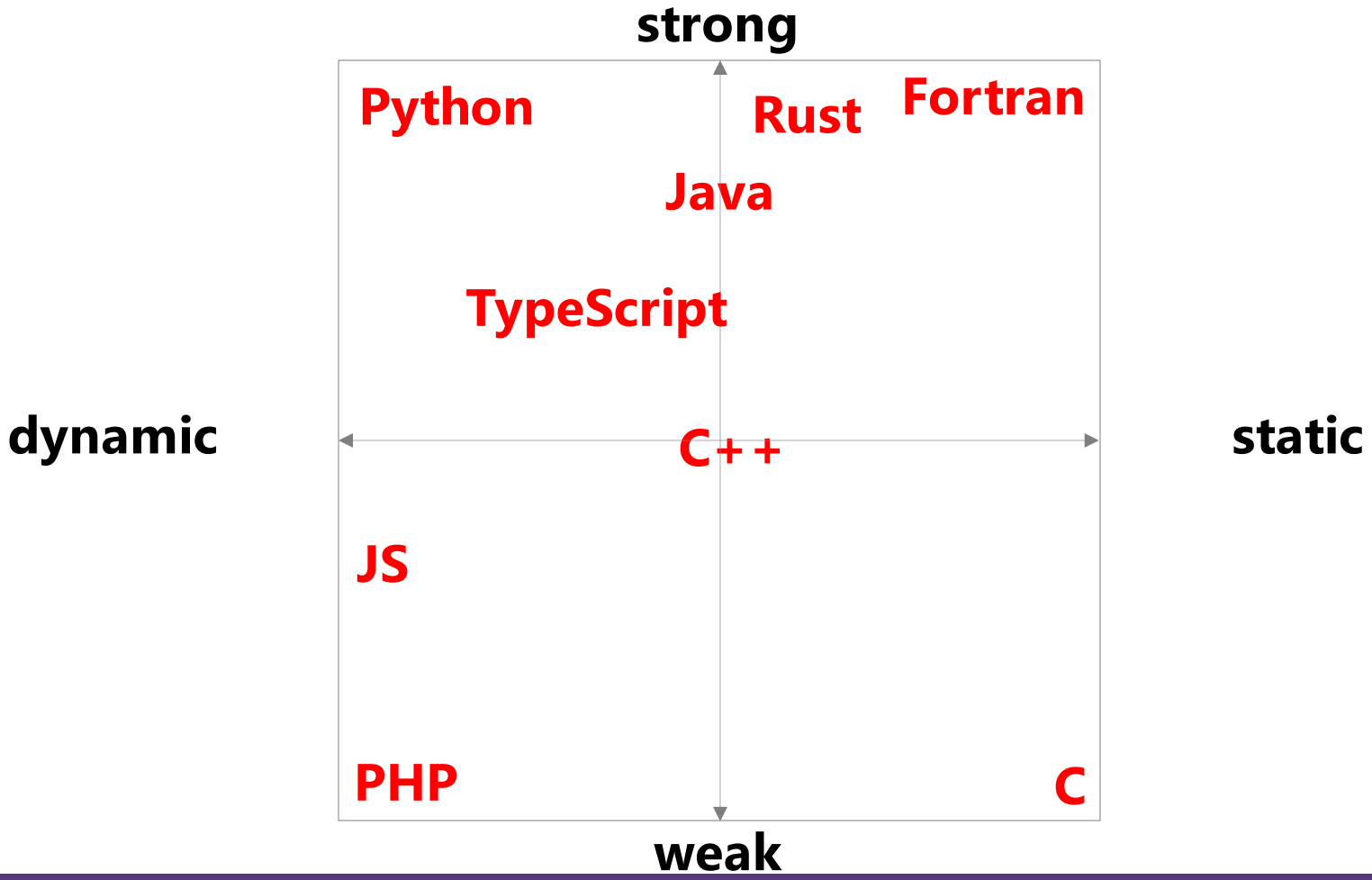
another definition is that weakly-typed languages do **lots of implicit conversions** and almost never give **type errors,** instead choosing to “do something else.”

```javascript
// JavaScript again
alert([] + []); // ""
alert({} + {}); // NaN
alert({} + []); // 0
alert([] + {}); // {}
```

I like to think “weakly-typed” means “the language doesn’t care about types, for some value of ‘doesn’t care’” lol

The typing alignment chart

- if we treat strong/weak and static/dynamic as axes on a spectrum…



Type inference

- **type inference** *automatically* determines what type something is.
- you may not know it, but you've been using it for years.

```
int x = 10;
int y = x + 5;
```

the compiler **implicitly** determines the types of the parts of the expressions, and **which addition operation** to do.

imagine having to write this...

```
int x = 10:int;
int y = x:int +:int 5:int;
```

type inference can go further, too, like variables in Rust!

```
let x = 10;       // x: i32
let y = Some(x); // y: Option<i32>
```

Defining Type Systems

Terms and types

- a **term** is a thing that **has a value.**
  - it's either a value itself, or it can be *evaluated* to get one.
    - (that sounds a lot like an **expression...**)
- every value **has a type,** and therefore every term **has a type.**

PL theory writes this relation like so:

`term:Type`

like:

`x:i32`

and now you know why Rust uses this syntax.

Inference rules

- a type system's **inference rules** explain how to give a type to a term.
- some rules are *axiomatic:* they are the "ground truth" of the types.
- others say how to determine the type of a **more complex** term.

here's an axiomatic rule that says **all integer literals** are of type **int:**

$$\texttt{<IntExp>:int}$$

(where **`IntExp`** is the kind of AST node that represents integer literals.)

and here's a rule for integer addition:

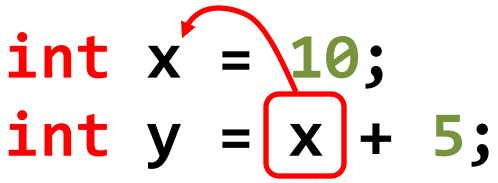
$$\frac{\texttt{t:int} \quad \texttt{u:int}}{\texttt{(t + u):int}}$$

"**if** t is an int and u is an int, **then** (t+u) is an int."

our type checking algorithm will be driven by these rules.
if no rule can be applied, **it's a type error!**

The type context

- in addition to the axiomatic rules, we also need some *context* to assign types to some terms, like **variables.**



how do we know what type x is?
we have to **look back to its declaration.**

fortunately, **name resolution** has already determined which symbol **x** refers to!

the **type context** is another mapping: it associates **symbols in the symbol table** with **types.**

here, `int` **x** *explicitly* associates **x** with the type `int`**...**

but in languages with **type inference,** the type of the initializer would have to be determined first.

Truss's type system

- let's examine the type system of this toy language to get familiar with how these concepts play out.
- first, it has a set of four **primitive types.**
  - `int`: a signed 32-bit integer
  - `bool`: a truth value, true or false
  - `string`: an immutable sequence of 0 or more codepoints
  - `()`: the absence of a type, called **void**
    - this is only used for function return types.
- it might not look like it, but we've sort of implicitly defined **the sets of valid values** for each type.
  - there are 2^{32} valid values for `int`...
  - 2 valid values for `bool`...
  - and so on.

Function Types

- any language that has functions will also need a type for them.
- or to be more accurate, **a type constructor** for them.

Truss writes its function types like: `fn(A₀, A₁, …): R`,
where `A₀`, `A₁` etc. are the arguments and `R` is the return type.

functions can have **any number** of arguments, which is why the argument types are written like that.

here are some valid function types:

```
fn(): ()
fn(int): int
fn(string, string): bool
fn(fn():()): ()
```

the last one is a **higher order function:** it takes a function as an argument!

Truss also has structs, but we'll come back for those later...

Inference rules

- we need rules for **every operation in the language.** examples:

$$\frac{\texttt{t:int} \quad \texttt{u:int}}{\texttt{(t } \textit{op} \texttt{ u):int}}$$ for any **op** in { `+`, `-`, `*`, `/`, `%` }

$$\frac{\texttt{t:string} \quad \texttt{u:string}}{\texttt{(t + u):string}}$$ (this is for string concatenation!)

$$\frac{\texttt{t:int} \quad \texttt{u:int}}{\texttt{(t } \textit{op} \texttt{ u):bool}}$$ for any **op** in { `==`, `!=`, `<`, `<=`, `>`, `>=` }

$$\frac{\texttt{t:bool} \quad \texttt{u:bool}}{\texttt{(t } \textit{op} \texttt{ u):bool}}$$ for any **op** in { `and`, `or` }

$$\frac{\texttt{f:fn(}A_0\texttt{,}A_1\texttt{,...):R} \quad \texttt{a}_0\texttt{:}A_0 \quad \texttt{a}_1\texttt{:}A_1 \quad \ldots}{\texttt{f(a}_0\texttt{, a}_1\texttt{, ...):R}}$$

this says, "the type of a function call is the function's return type."

What about statements?

- **statements** have gotten left out of the discussion, because they don't evaluate to a value, and therefore **have no type.**
- *but* statements *can* interact with the type system nonetheless.

what type does the condition have to be, in Java?

```
if(x < 10)
  println("yep");
```

boolean.

similar things apply for loops, **switch**es (and Rust **match**es), **return** statements, and so on.

(we *could* also say that statements have a type like **()** or **void**, but it isn't strictly necessary to do this.)

Implementation Details

Data structures needed

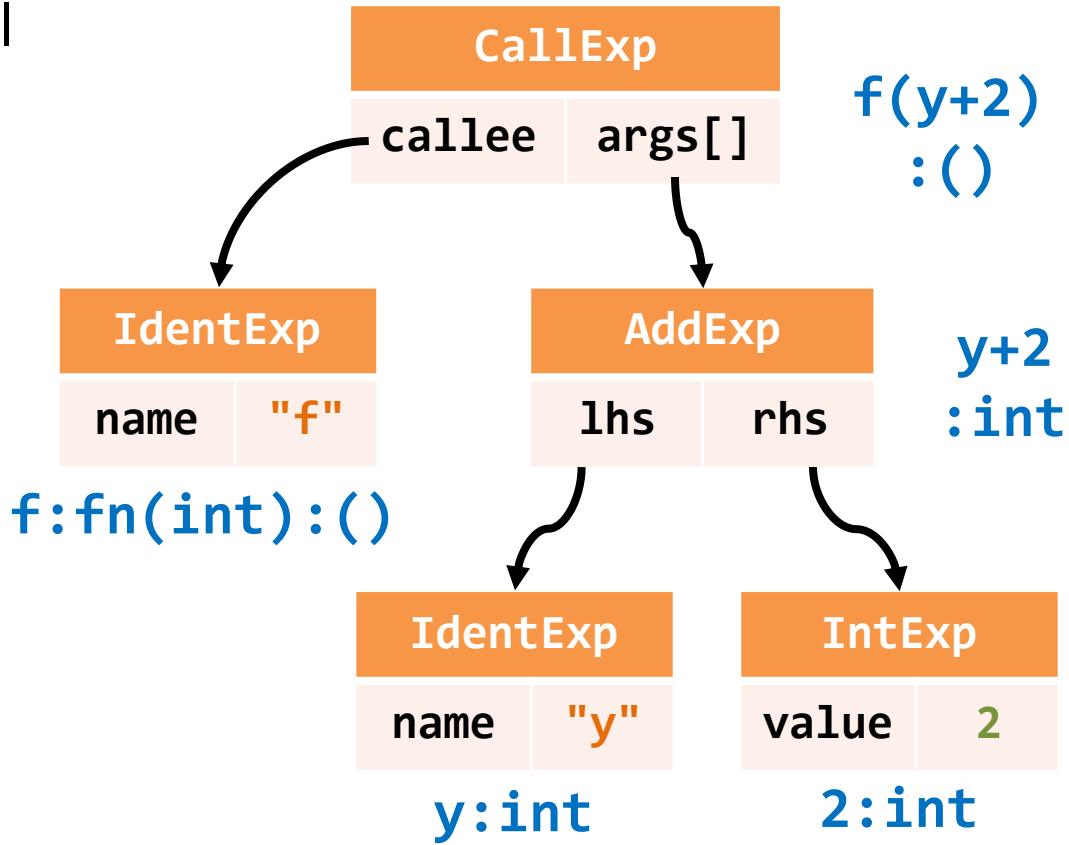
- we'll need to represent **types** in the compiler.
  - this can be as simple as a Rust **enum,** very similar to how we represent the kinds of AST nodes.
- we'll also need the **type context** like we said before.
  - we said we needed this to map from ***symbols* to types...**
  - but we're going to extend this to ***all AST nodes* that can have types,** because subsequent compiler passes will need this info.
  - this will get filled in as we do type checking.
- finally, we need a way to represent the **terms...**
  - wait, don't we have that already?

OH, right, the AST!

- the AST encodes terms as **trees.**
- this implies a relatively straightforward way to check for types:

1. do a **depth-first** traversal to typecheck the **leaves,** using the axiomatic rules and type context;

2. typecheck the internal nodes **according to the types of their children,** using the inference rules.



Implementing the inference rules

- the inference rules are easier to implement than they might seem.
- most of the checks boil down to: **is this type equal to that type?**
- for instance in this rule:
  - $\frac{\texttt{t:int} \quad \texttt{u:int}}{\texttt{(t } \textit{op} \texttt{ u):bool}}$ for any **op** in { `==`, `!=`, `<`, `<=`, `>`, `>=` }
- if I am looking at an AST node for a `==`, the check goes like this:
  1. check that LHS's type equals `int`, and give an error if not.
  2. check that RHS's type equals `int`, and give an error if not.
  3. set this `==` AST node's type to `bool` in the type context.
- that's it. some of the rules are more involved, but it's rarely much more complex than just checking type equality.